promptToPpt

Master Technical Specification & Claude Code Build Prompt

Prompt → Research → Story → Slides → Visual Design → Browser Editor → Collaboration → Export

Purpose	A production-grade AI presentation SaaS inspired by the interaction patterns of modern AI deck builders.
Primary product	Users enter a topic/prompt and receive an editable, visually rich presentation.
Public web	Next.js + TypeScript for SEO-focused marketing, pricing, templates, blog, documentation and acquisition pages.
Application	React + TypeScript editor on a separate app surface/subdomain, connected to the same backend and identity system.
Infrastructure target	AWS-first, cost-conscious, scalable architecture; exact services must be justified by Claude after research.
AI	Development: Gemini API where practical. Production: provider abstraction with OpenRouter or another justified gateway; image generation included.

Instruction: Treat this document as the master product/engineering brief. Do not start by generating the entire application in one uncontrolled pass. First inspect the repository, Figma, reference screenshots and constraints; then produce an implementation plan, architecture decisions and milestones before coding.

Table of Contents

1. Master instruction to Claude Code
2. Product vision and non-negotiable goals
3. Reference screenshots and editor UX
4. User journeys and application flow
5. Prompt-to-PPT intelligence and clarification flow
6. Input/reference sources
7. Presentation generation pipeline
8. Slide design system and visual quality
9. Browser editor requirements
10. Public website and Next.js SEO architecture
11. React editor architecture
12. Backend, database, storage and job processing
13. AI provider strategy and cost control
14. Token/model usage analytics — mandatory owner dashboard
15. Authentication, billing, plans and country pricing
16. Sharing, presenting and export
17. Analytics, SEO and product telemetry
18. Security, privacy, abuse controls and reliability
19. AWS deployment and environments
20. Repository structure and developer workflow
21. Implementation phases and acceptance criteria
22. Required technical decisions Claude must research
23. Copy/paste master prompt

3. Reference screenshots and editor UX

The two supplied screenshots are visual references for the editor experience. Recreate the interaction patterns and information hierarchy, but do not copy proprietary source code, assets or exact brand identity. The goal is a functionally comparable, original implementation using the product's own design system.

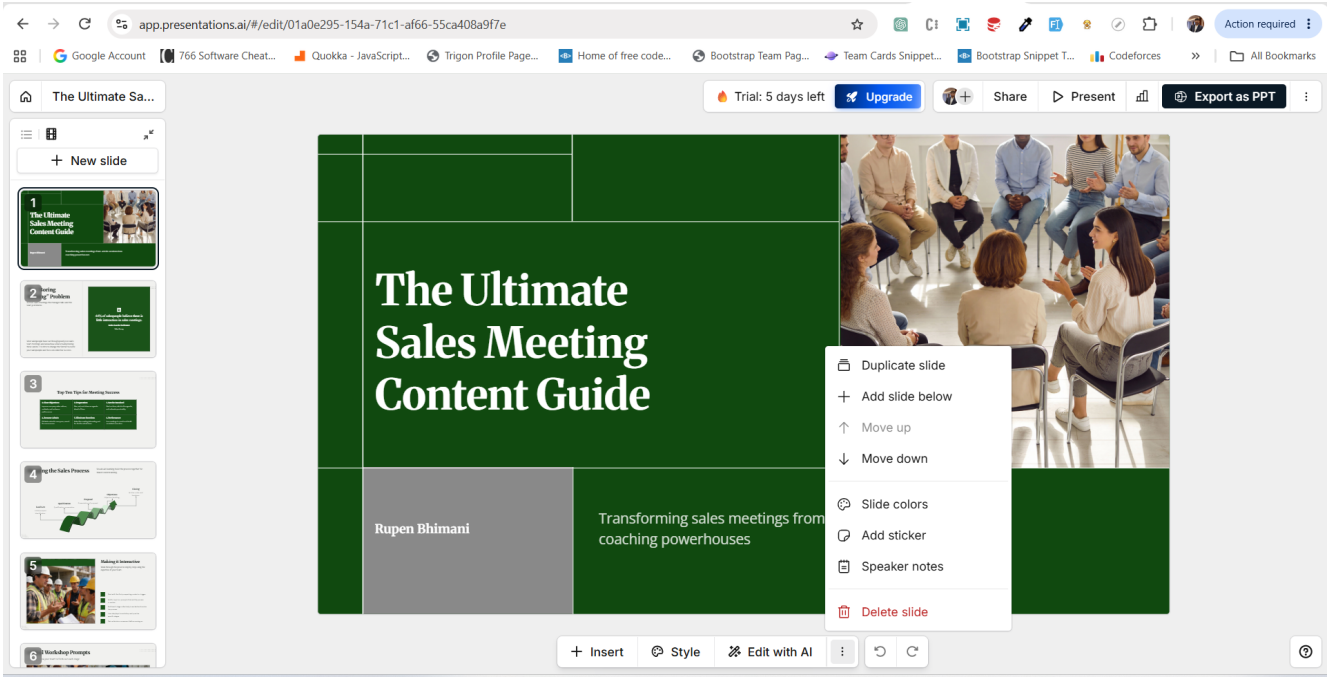


Figure 1. Supplied reference screenshot 1.

- Left rail: slide thumbnails and slide numbering.
- Main canvas: editable presentation with image/text layout.
- Top-right: share, present and PPT export actions.
- Context menu: duplicate, add below, move, slide colors, sticker, speaker notes and delete.
- Bottom toolbar: insert, style, AI editing, overflow actions, undo and redo.

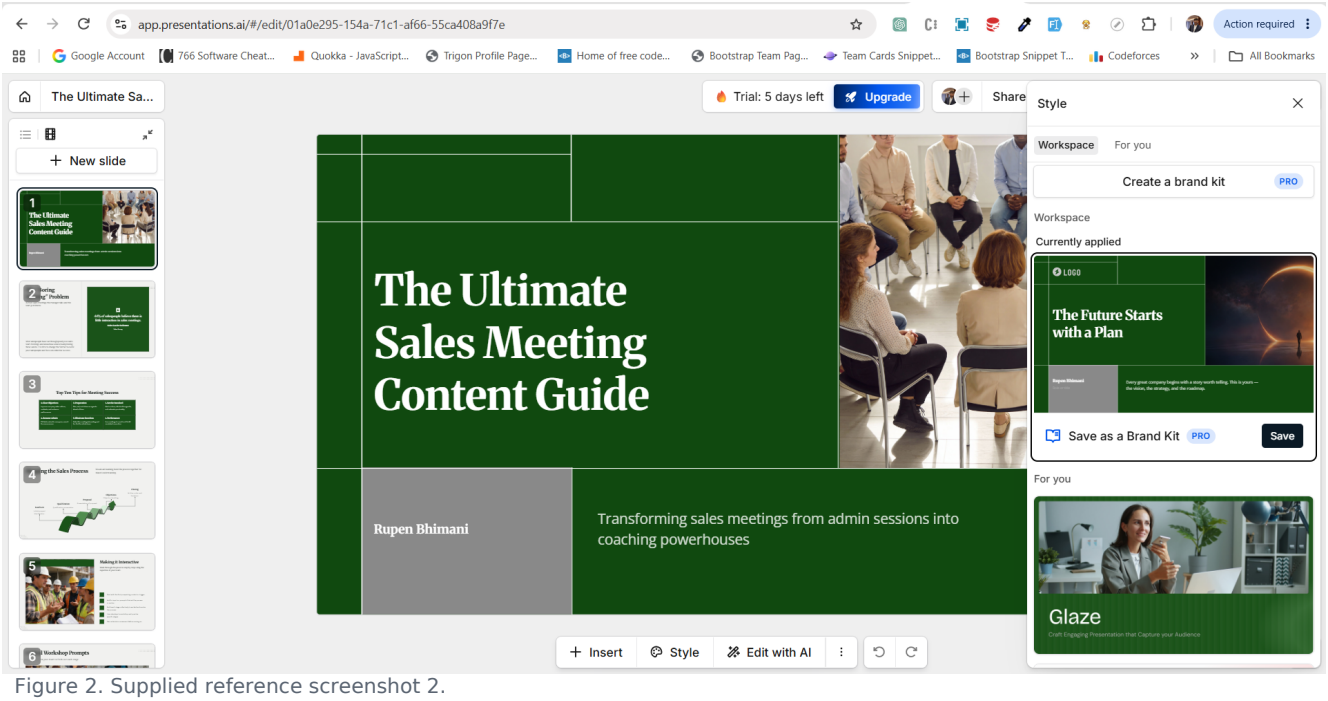


Figure 2. Supplied reference screenshot 2.

- Style drawer: workspace styles, recommended styles and a brand-kit entry point.

- Users should be able to change slide design/theme from a large collection of compatible layouts.
- Brand kit is a higher-tier feature and should be feature-flagged for a later phase.
- The design system must remain coherent when the user changes templates, colors or typography.

1. Master instruction to Claude Code

Core operating instruction

- You are the principal SaaS architect, staff full-stack engineer, AI systems engineer, UI engineer and DevOps engineer responsible for building promptToPpt.
- Do not rush into implementation. First inspect the existing repository, package managers, environment files, Figma link(s), supplied screenshots and any existing code. Produce a short discovery report and implementation plan before changing architecture.
- Do not invent missing product behavior. Where the specification is ambiguous, research the current market and document the decision. Prefer simple, maintainable and cost-controlled infrastructure.
- Build production-quality TypeScript. Avoid throwaway prototypes when the component is part of the long-term architecture.
- Keep public SEO pages and the interactive presentation editor as separate applications/folders while sharing contracts, authentication, APIs and data models cleanly.
- Every major decision must be recorded in an ADR (Architecture Decision Record) with: problem, options, decision, cost/performance implications, trade-offs and migration path.

Definition of done for Claude

- Repository is installable from a clean machine.
- Both applications run locally with documented commands.
- Development and production environment variables are separated and validated.
- AI generation is asynchronous and observable.
- Every AI request records provider/model, input tokens, output tokens, cached/reasoning/image usage where available, latency, status, user, presentation and cost.
- Presentation content is stored as structured JSON, not only as rendered HTML/canvas state.
- Every slide is independently editable and exportable.
- Undo/redo, autosave and version recovery work.
- Owner analytics can calculate product usage and AI spend independently of the AI provider dashboard.
- All paid actions are server-authorized; never trust client-side plan flags.
- Tests cover the critical generation, billing, permissions, editor state and export paths.

2. Product vision and non-negotiable goals

Product

- Name: promptToPpt.
- Core promise: enter a topic or detailed prompt, optionally provide source material, and generate a high-quality editable presentation.
- The experience should feel like an AI presentation workspace rather than a simple text-to-PPT converter.
- Visual quality is a core product requirement: layouts, typography, images, icons, charts, diagrams, spacing, hierarchy and theme consistency must be treated as first-class outputs.

Input depth behavior

- If the user enters only a broad topic, do not immediately make an arbitrary long deck. First infer useful directions and show 3–4 concise topic/angle options.

- Example: user enters Salesforce Agentforce. The system can propose options such as overview, architecture/how it works, business use cases, implementation/ROI, or another researched angle. Let the user select one or combine them.
- If the user supplies detailed instructions, audience, slide count, chapters, tone or structure, respect those instructions and minimize unnecessary clarification.
- If important information is missing, ask targeted questions such as audience, objective, slide count, presentation duration, tone, desired chapters and source priority.
- Never trap the user in a long questionnaire. Ask only what materially improves the deck.

Quality principles

- Prefer evidence-backed content when references are supplied.
- Preserve citations/source traceability for sourced claims where possible.
- Do not fabricate statistics, quotations, customer logos, research findings or factual claims.
- Use image search/stock/image-generation strategy based on licensing and product policy.
- Generate layouts that adapt to content length; avoid overflow, clipping and tiny text.
- Every generated deck must pass automated structural and visual validation before being marked complete.

4. User journeys and application flow

A. Public acquisition flow

- Landing page → explain value → show examples/templates → CTA → authentication → app.
- Public pages: Home, AI PPT Maker, AI Presentation Maker, PPT Generator, Templates, Pricing, Blog, Contact, FAQ, About/Trust, Terms, Privacy and SEO landing pages.
- Use the provided Figma design as the visual source of truth for the public site.

B. New presentation flow

- 1. User opens the application.
- 2. User enters a prompt/topic.
- 3. User can attach/import references.
- 4. System classifies prompt depth.
- 5. Broad prompt → propose 3–4 directions and ask for a selection.
- 6. Detailed prompt → extract requirements and proceed.
- 7. Reference material → ingest, parse, chunk, rank and cite.
- 8. Generate outline.
- 9. Let user confirm/edit outline when useful.
- 10. Generate slide content + layout plan.
- 11. Generate/find visual assets.
- 12. Render slide previews.
- 13. Run validation.
- 14. Open editor with generated deck.
- 15. Autosave continuously.
- 16. User edits manually or with AI.
- 17. Share/present/export depending on plan.

C. Returning user

- Dashboard should show recent presentations, drafts, folders/projects, templates, usage/credits and account status.
- Opening a presentation should restore the latest saved version.
- Autosave must be resilient to refreshes, network interruption and multiple tabs.

5. Prompt-to-PPT intelligence and clarification flow

Prompt analysis

- Create a structured prompt-analysis object with fields such as: topic, objective, audience, tone, language, slideCount, duration, requiredSections, sourceMaterials, visualStyle, brandConstraints and outputFormat.
- Detect whether the user has provided enough information to generate a useful deck.
- Use a deterministic policy layer around the model so model output cannot bypass product rules.
- Do not let the model directly write arbitrary database records; use validated schemas and server-side tools.

Suggested broad-topic response

- Topic: Salesforce Agentforce
- Show 3–4 selectable cards/buttons:
- • Executive overview
- • How Agentforce works / architecture
- • Business use cases and ROI
- • Implementation roadmap / adoption
- After selection, ask whether the user wants a short, standard or detailed deck, or infer a sensible default.

Outline-first generation

- Generate an outline before expensive visual generation.
- Each slide object should contain: id, title, purpose, content blocks, speaker notes, sources, layoutId, visual requirements, design tokens and validation status.
- Allow outline regeneration without losing the original prompt.
- Allow the user to lock slides or sections so later AI edits cannot change them.

AI editing

- Support commands such as rewrite, shorten, expand, change tone, translate, add slide, remove slide, reorder, improve visual hierarchy, create chart, create diagram, replace image and restyle.
- AI edits should operate on structured slide data and return a validated patch, not a whole uncontrolled deck.

6. Input/reference sources

Supported sources

- PDF upload.
- PPT/PPTX upload.

- Website/blog URL.
- YouTube URL.
- Audio upload or audio-to-PPT workflow.
- Pitch deck/reference deck.
- Multiple references in one generation.
- Text pasted directly by the user.
- Future: cloud-storage connectors.

Ingestion pipeline

- Upload → virus/type/size validation → object storage → extraction → normalization → chunking → metadata → embeddings/retrieval if justified → source citations → generation context.
- For PDF/PPT, extract text and structural information; retain page/slide references.
- For YouTube, obtain transcript when legally and technically permitted; preserve source URL and timestamps where available.
- For web pages, extract main content rather than navigation/ads where possible.
- For audio, transcribe first, then summarize/structure into a presentation.
- Keep raw files private and signed URLs short-lived.

Source priority rules

- Explicit user instructions > selected outline > high-confidence user references > general model knowledge.
- When sources conflict, surface the conflict or prefer the most authoritative/recent source rather than silently merging claims.

7. Presentation generation pipeline

Recommended pipeline

- Stage 1 — request validation.
- Stage 2 — prompt/source analysis.
- Stage 3 — research/context retrieval if requested.
- Stage 4 — outline generation.
- Stage 5 — slide content generation.
- Stage 6 — layout selection.
- Stage 7 — visual asset selection/generation.
- Stage 8 — composition/rendering.
- Stage 9 — quality validation.
- Stage 10 — persistence/version creation.
- Stage 11 — editor handoff.
- Stage 12 — usage/cost accounting.

Asynchronous jobs

- Generation must not depend on one long HTTP request.
- Use a job/queue architecture with states such as queued, analyzing, outlining, generating, rendering, validating, completed, failed, cancelled.
- Support retries with idempotency keys.

- Expose progress to the UI via polling, SSE or WebSocket based on the chosen architecture.
- Store stage-level timing and model cost.

Validation

- Schema validation.
- Missing-content validation.
- Text overflow estimation.
- Image aspect-ratio validation.
- Duplicate-layout detection.
- Contrast/accessibility checks.
- Unsupported-font fallback detection.
- Export render smoke test.
- Final deck completeness score.

8. Slide design system and visual quality

Theme engine

- Create a design-token system: primary/secondary/accent colors, background, surface, text colors, typography scale, font families, border radius, shadows, spacing, grid and chart styles.
- Support per-presentation theme plus per-slide overrides.
- Support user-defined fonts where licensing/browser support allows.
- Allow users to define font family, heading/body typography and color palette.
- Brand kit is a later Pro/paid capability; implement behind a feature flag and data model now, but do not block MVP.

Layout library

- Create reusable semantic layouts: title, title+body, split content, image+text, quote, timeline, process, comparison, metrics, chart, table, roadmap, architecture, case study, section divider, thank-you and blank canvas.
- Layouts must be responsive within the fixed slide aspect ratio.
- Choose layout based on content shape, not randomly.
- Allow users to swap a slide to another compatible design without losing content.
- Right-side style/design drawer should show alternative slide designs/themes.

Visual assets

- Use images, icons, illustrations, charts and diagrams where they improve comprehension.
- Do not decorate every slide unnecessarily. Visuals must support the message.
- Keep image licensing/source metadata.
- Image generation should be provider-abstracted and usage-metered.
- Use placeholders only during generation; resolve them before completion or clearly mark the slide as still processing.

Animation

- Editor may store animation metadata, but PPTX export must degrade gracefully if a requested animation is not supported.
- Present mode can support web-native transitions/animations independent of PPTX.

9. Browser editor requirements

Layout

- Left sidebar: slide thumbnails, slide numbers, add slide.
- Center: presentation canvas.
- Right sidebar: style/design/AI panels.
- Top-right: share, present, export.
- Bottom toolbar: insert, style, AI edit, overflow, undo and redo.
- Responsive behavior for smaller screens; editor is desktop-first.

Slide interactions

- Context menu: duplicate slide, add slide below, move up/down, slide colors, add sticker, speaker notes, delete slide.
- Add slide button should offer blank, layout, template and AI-generated slide options.
- Drag/reorder thumbnails.
- Duplicate should preserve slide data but create new IDs.
- Delete should support undo.
- Speaker notes are stored separately from visible slide content.

Editing

- Inline text editing with selection, rich text controls, alignment, lists and links as appropriate.
- Object-level selection for images, shapes, icons, charts and text blocks.
- Resize/move with snapping and guides.
- Lock/unlock objects.
- Duplicate, copy/paste and delete.
- Keyboard shortcuts.
- Undo/redo must be command-based or state-diff based, not full page reloads.

Style drawer

- Workspace styles.
- Recommended/For You styles.
- Alternative slide designs.
- Theme colors.
- Typography.
- Backgrounds.
- Image treatment.
- Chart style.
- Brand kit (paid/future).

Editor state

- Use a normalized presentation state and immutable update patterns.
- Separate document state from transient UI state.
- Persist only serializable document state.
- Support optimistic UI with conflict-safe autosave.

10. Public website and Next.js SEO architecture

Required split

- Create two applications/folders in one repository: web for Next.js and app for React editor.
- Main domain uses Next.js for SEO and acquisition.
- Application/editor runs on a separate subdomain such as app..
- Both share authentication, database APIs, design tokens, types and service contracts without duplicating business logic.

Next.js pages

- Home.
- AI PPT Maker.
- AI Presentation Maker.
- PPT Generator.
- Text to PPT.
- PDF to PPT.
- Templates.
- Template detail pages.
- Pricing.
- Blog index and blog detail.
- Contact.
- FAQ.
- About/Trust.
- Privacy, Terms, Refund/Cancellation.
- Authenticated account pages only if they benefit from server rendering.

SEO requirements

- Server-rendered/static pages where possible.
- Metadata API, canonical URLs, Open Graph/Twitter cards, sitemap, robots.txt, structured data.
- Programmatic SEO for template/use-case pages only when pages have genuinely useful unique content.
- Fast LCP/INP/CLS and optimized image delivery.
- Do not expose private presentation content to search engines.
- Use semantic HTML and accessible headings.

Figma integration

- The user will provide a Figma link. Treat it as the visual source of truth for the landing/public site.
- Before coding, inspect typography, spacing, colors, responsive breakpoints, components and assets.
- Create reusable design tokens/components instead of one-off page CSS.

11. React editor architecture

Technology

- React + TypeScript.
- Use a proven state-management approach appropriate for a large editor; Claude must compare Zustand, Redux Toolkit and alternatives before selecting.
- Use a component library only where it does not fight the custom design system.
- Prefer a rendering strategy that makes precise positioning, text editing and export feasible.
- Do not introduce a canvas technology just because it is popular; compare DOM/SVG/canvas/hybrid approaches against the actual editor requirements.

Shared packages

- packages/types — shared TypeScript contracts.
- packages/ui — shared design system where practical.
- packages/theme — theme tokens and layout definitions.
- packages/api-client — typed API client.
- packages/validation — Zod or equivalent schemas.
- packages/config — lint/format/build configuration.
- Keep editor-specific code inside the React app.

Performance

- Virtualize slide thumbnails if the deck becomes large.
- Lazy-load heavy editor panels.
- Debounce autosave.
- Use image thumbnails/proxies in the editor.
- Keep large source documents out of React state.
- Measure render time and interaction latency.

12. Backend, database, storage and job processing

Decision requirement

- Claude must evaluate the backend options instead of blindly choosing one. Compare managed PostgreSQL, AWS RDS/Aurora PostgreSQL, serverless PostgreSQL, DynamoDB and BaaS options for this workload.
- Primary criteria: total cost at 1k, 10k, 100k and 1M users; operational complexity; query flexibility; transaction support; analytics; backups; scaling; latency; vendor lock-in.
- Prefer PostgreSQL if it is the best fit because the product has relational entities, billing, usage ledgers, presentations, versions, permissions and analytics.
- Use S3 or equivalent object storage for uploaded files, generated assets and exports.

Core entities

- User.
- Organization/Workspace (future/team support).
- Subscription.
- Plan.
- CreditBalance/Ledger.
- Presentation.

- PresentationVersion.
- Slide.
- SlideAsset.
- SourceDocument.
- SourceChunk.
- GenerationJob.
- AIRequest.
- AIUsageEvent.
- ModelPricingSnapshot.
- ImageGenerationRequest.
- ExportJob.
- ShareLink.
- Template.
- Theme.
- BrandKit.
- Payment.
- Invoice/Transaction.
- AnalyticsEvent.
- AuditLog.

Usage ledger

- Never calculate billing from the current provider dashboard only.
- Record every AI operation in an immutable usage ledger.
- Store provider, model, model version, request type, input tokens, cached tokens, output tokens, reasoning tokens if available, image count/megapixels if applicable, latency, provider-reported cost, platform markup, user credit cost and status.
- Maintain a pricing snapshot table so historical costs remain reproducible after providers change prices.

File lifecycle

- Uploaded file → quarantine/validation → S3 private bucket → parsed representation → source index → retention policy.
- Generated PPTX/PDF → private storage → signed download URL.
- Failed/abandoned artifacts should have cleanup policies.

13. AI provider strategy and cost control

Development

- Use the user's available Gemini API during development/testing where permitted by the account and API terms.
- Keep the provider behind an interface so no application feature directly depends on Gemini-specific response shapes.
- Never commit API keys.
- Development usage must still be logged into the same AI usage tables as production.

Production

- Evaluate OpenRouter and direct providers. OpenRouter is attractive for model switching and unified API access, but Claude must calculate the platform fee, provider pricing and latency trade-offs before finalizing.
- Create provider adapters such as GeminiProvider, OpenRouterProvider and DirectProvider.
- Implement a ModelRegistry containing model capabilities: text, vision, structured output, tool use, image generation, context window, estimated cost and quality tier.
- Use different models for different jobs rather than one expensive model for everything.

Suggested model-routing classes

- Fast/cheap: classification, cleanup, small rewrites.
- Reasoning/quality: outline, complex synthesis, slide narrative.
- Vision: PDF/PPT/image understanding.
- Image generation: presentation visuals.
- Embeddings/retrieval: source search if RAG is selected.
- Export/validation: deterministic tools, not an LLM where avoidable.

Cost controls

- Per-user daily/monthly generation limits.
- Plan-level credit budgets.
- Maximum slide count per generation.
- Maximum source size/pages/minutes.
- Model fallback rules.
- Request timeouts and retry limits.
- Cache repeated analysis where safe.
- Do not regenerate unchanged slides.
- Show estimated credit consumption before expensive operations where practical.

Image generation

- Evaluate OpenRouter's image generation API and direct image providers.
- Compare cost per image, quality, aspect-ratio support, latency, commercial terms and reference-image support.
- Allow a per-slide visual budget.
- Prefer generated images only when they materially improve the slide; otherwise use icons/charts/stock sources.

14. Token/model usage analytics — mandatory owner dashboard

Core requirement

- This is a must-have, not a future enhancement. The product owner must be able to understand AI cost and user behavior in both development and production.
- Every model call must produce a normalized AIUsageEvent before the request is considered complete.
- Dashboard data must be independent enough to audit against provider invoices.

Owner dashboard navigation

- Overview.
- Sales & revenue.
- Subscriptions.
- AI usage.
- Model usage.
- Token usage.
- Image generation usage.
- Cost & margin.
- Presentations generated.
- Exports.
- Users.
- Countries.
- Top prompts/use cases.
- Errors & failed generations.
- Latency/performance.
- Provider health.
- Audit logs.
- Feature flags.

Required filters

- Today / yesterday / 7 days / 30 days / month / quarter / year / custom date range.
- Country.
- Plan.
- User.
- Provider.
- Model.
- Request type.
- Presentation.
- Success/failure.
- Development/staging/production environment.

Required metrics

- Total input tokens.
- Total output tokens.
- Cached tokens.
- Reasoning tokens where available.
- Total model calls.
- Average tokens per deck.
- Median/95th percentile generation time.
- AI cost.
- Image generation count and cost.
- Cost per presentation.
- Cost per slide.
- Revenue per presentation.
- Gross margin estimate.

- Credits consumed.
- Unused credits.
- Failure rate.
- Retry rate.
- Model fallback rate.

Graphs

- Token usage over time.
- AI cost over time.
- Cost by model.
- Token distribution by model.
- Cost by user/plan/country.
- Generation volume by day/week/month.
- Revenue vs AI cost.
- Gross margin trend.
- Top 20 models by spend.
- Top 20 users by usage.
- Top generation failures.

Alerting

- Daily AI spend threshold.
- Unexpected model price increase.
- Sudden token spike.
- Provider error-rate spike.
- Per-user abuse threshold.
- Margin below configured threshold.

15. Authentication, billing, plans and country pricing

Authentication

- Support email/password and at least one social login after selecting the auth provider.
- Use secure sessions/cookies; never store long-lived secrets in localStorage.
- Roles: user, owner/admin, support/admin as needed.
- Presentation permissions should be explicit.

Plans

- Use three customer plans initially: Free, Pro and a higher-tier plan.
- During development/testing, use the current Presentations.AI reference pricing/features as the benchmark, but do not hard-code third-party pricing. Claude must live-check the official pricing page before implementation and document the date/source.
- Current official Presentations.AI sources are not fully consistent: the official pricing page currently shows Starter $0, Pro $20/month billed annually and Gold $100/month billed annually with 50,000 credits, while another official product page shows Gold at $50/month with 25,000 credits. Treat this as a live-verification requirement, not as a fixed fact.

- Design the pricing engine so plan prices, credits, limits and feature flags are database-configurable.

India vs international pricing

- India: display INR.
- Outside India: display USD initially.
- Determine country server-side from a trusted signal and allow the user to see the billing currency before checkout.
- Do not trust client-side country/currency values for payment authorization.
- Use Razorpay for India and evaluate Stripe or another international processor for non-India payments; payment providers must be abstracted behind a billing service.
- Handle taxes/GST, invoices, refunds, webhook verification and subscription state transitions.

Credits

- Define a normalized internal credit unit independent of raw provider tokens.
- Every plan maps to credits plus hard limits.
- AI operations consume credits according to a configurable cost table.
- Keep raw token cost and customer credit cost separately.
- Prevent negative balances with atomic server-side transactions.

16. Sharing, presenting and export

Share

- Generate secure share links with view/edit permissions.
- Optional password protection and expiration for higher tiers.
- Owner can revoke a share link.
- Never expose private source files through a public presentation URL.

Present mode

- Full-screen browser presentation.
- Keyboard navigation.
- Slide transitions.
- Speaker notes view for presenter.
- Optional audience link/remote controls as a future phase.

Export

- Export to PPTX.
- Export to PDF.
- Optional PNG/JPG slide export.
- Export jobs should be asynchronous for large decks.
- Generated PPTX must preserve editable text and common shapes where feasible.
- Font substitution must be explicit.
- Unsupported editor features must have graceful fallbacks.

17. Analytics, SEO and product telemetry

Web analytics

- Integrate Google Analytics 4 or an equivalent analytics stack.
- Track acquisition pages, CTA clicks, signup, prompt submission, generation start, generation completion, editor usage, export, upgrade and cancellation.
- Do not send private prompt contents, uploaded document contents or sensitive presentation data to analytics vendors.

Product events

- signup_completed
- prompt_submitted
- clarification_shown
- clarification_completed
- source_uploaded
- generation_started
- generation_completed
- generation_failed
- slide_added
- slide_deleted
- slide_duplicated
- slide_reordered
- ai_edit_used
- theme_changed
- template_changed
- share_created
- presentation_started
- export_started
- export_completed
- paywall_viewed
- checkout_started
- subscription_started
- subscription_cancelled

SEO/content

- Blog should be database/CMS driven and support metadata, author, publish date, categories, internal links and schema.
- Templates should have indexable detail pages with unique copy, use cases and examples.
- Build an internal linking strategy around AI PPT maker, AI presentation maker, PowerPoint generator, PPT generator, text-to-PPT and related use cases.

18. Security, privacy, abuse controls and reliability

Security

- Validate every request server-side.

- Rate-limit authentication, generation, upload and export endpoints.
- Use signed URLs for private assets.
- Encrypt data in transit and at rest.
- Secrets only in server-side secret stores/environment variables.
- Audit admin actions.
- Verify payment webhooks cryptographically.
- Use CSRF protections where applicable.
- Use strict content security policy where feasible.

AI safety/abuse

- Limit prompt size and upload size.
- Block obviously malicious file types and malformed payloads.
- Protect against prompt injection from uploaded documents: source content is data, not system instructions.
- Separate trusted system instructions from retrieved content.
- Do not let retrieved content execute tools without explicit policy checks.
- Implement abuse quotas and anomaly detection.

Reliability

- Idempotent jobs.
- Retry with exponential backoff.
- Dead-letter queue.
- Health checks.
- Provider fallback.
- Graceful degradation when image generation fails.
- Autosave recovery.
- Database backups and restore testing.

19. AWS deployment and environments

Environment model

- Local development.
- Development/staging.
- Production.
- Never share production secrets with local development.

AWS evaluation

- Claude must compare ECS/Fargate, Lambda, EC2 and other appropriate compute choices for API and worker workloads.
- Evaluate managed PostgreSQL options for cost at low, medium and high usage.
- Evaluate S3 + CloudFront for asset delivery.
- Evaluate SQS for generation jobs.
- Evaluate CloudWatch/OpenTelemetry for logs and traces.
- Evaluate Secrets Manager/Parameter Store for secrets.
- Evaluate WAF/rate limiting where justified.

- Choose the minimum operational complexity that still supports the expected product.

Cost model

- Create a spreadsheet/markdown cost model for 1k, 10k, 100k and 1M monthly active users.
- Include compute, database, object storage, bandwidth, queue, logs, monitoring, email, payment fees and AI provider costs.
- Show fixed vs variable costs.
- Show cost per generated deck under realistic token/image assumptions.
- Use this model to inform pricing and plan limits.

20. Repository structure and developer workflow

Suggested monorepo

- apps/web — Next.js public website.
- apps/editor — React + TypeScript presentation editor.
- apps/api or services/api — backend API if a dedicated service is selected.
- workers/generation — async generation worker if needed.
- packages/types — shared types.
- packages/validation — shared schemas.
- packages/ui — shared UI primitives.
- packages/theme — presentation design tokens/layout definitions.
- packages/ai — provider abstraction and model registry.
- packages/billing — billing abstraction.
- packages/analytics — event definitions/client/server tracking.
- packages/config — shared lint/format/TS config.
- docs/adr — architecture decisions.
- docs/api — API contracts.
- docs/product — product specifications.

Claude Code workflow

- Phase 0: inspect and research.
- Phase 1: scaffold and shared contracts.
- Phase 2: auth/database/storage.
- Phase 3: prompt + generation pipeline.
- Phase 4: editor core.
- Phase 5: export/share/present.
- Phase 6: billing/credits.
- Phase 7: owner analytics.
- Phase 8: public website/SEO.
- Phase 9: hardening/testing/deployment.
- After each phase, run lint, typecheck, tests and a manual acceptance checklist before moving on.

Code quality

- TypeScript strict mode.
- ESLint + formatter.
- Schema validation.
- Unit tests for pure logic.
- Integration tests for API and jobs.
- End-to-end tests for the critical user journey.
- Visual regression tests for key editor states if practical.
- No unused environment variables, dead services or duplicate business logic.

21. Implementation phases and acceptance criteria

Phase 0 — discovery

- Inspect Figma and screenshots.
- Research current competitors and infrastructure pricing.
- Produce architecture decision record.
- Produce ERD and event model.
- Produce AI provider/model/cost matrix.
- Produce page map and editor wireframe.
- Do not implement until these artifacts are reviewed.

Phase 1 — foundation

- Monorepo, CI, env validation, shared types, database migration setup, auth skeleton, object storage, logging.

Phase 2 — generation MVP

- Prompt → analysis → outline → slide JSON → render → editor.
- Broad topic clarification flow.
- Basic PDF/PPT/web source ingestion.
- AI usage tracking.

Phase 3 — editor

- Slide thumbnails, canvas, text editing, layout switching, style drawer, context menu, undo/redo, autosave.

Phase 4 — export/share

- PPTX/PDF export, present mode, share links.

Phase 5 — monetization

- Plans, credits, paywall, India/international payments, invoices, webhooks.

Phase 6 — owner intelligence

- Full usage dashboard, model analytics, cost/margin analytics, sales dashboard, alerts.

Phase 7 — SEO/public site

- Figma implementation, SEO pages, blog, templates, pricing, analytics.

Phase 8 — hardening

- Security review, abuse controls, load tests, failure injection, backup restore test, production deployment.

MVP acceptance checklist

- A user can enter a simple topic and receive 3–4 useful directions.
- A user can provide a detailed prompt and generate a deck without unnecessary questions.
- A user can upload a reference and the deck reflects it.
- A user can edit every slide's text.
- A user can reorder, duplicate, add and delete slides.
- A user can change a slide design/theme.
- Undo/redo works.
- Autosave survives refresh.
- Usage tokens/cost are visible to the owner dashboard.
- Plan limits are enforced server-side.
- India and international pricing display correctly.
- PPTX export is usable.
- Public pages are indexable and fast.

22. Required technical decisions Claude must research

Research before implementation

- Best editor rendering architecture for an editable PPT-like web canvas.
- Best PPTX generation/export library for TypeScript and its fidelity limitations.
- Best PDF export approach.
- Best PPT/PDF parsing libraries.
- Best YouTube transcript ingestion approach and legal/technical constraints.
- Best audio transcription provider/cost.
- Best image generation provider and OpenRouter image model options.
- Gemini API capabilities and current billing.
- OpenRouter current model/provider fees and usage reporting.
- Direct Anthropic/OpenAI/Google provider economics vs OpenRouter.
- Best PostgreSQL hosting option for the expected workload.
- AWS compute choice for API + long-running generation workers.
- Queue choice.
- Redis/cache choice, only if actually needed.
- Auth provider.
- Payment provider split for India and international.
- Analytics and observability stack.
- Email provider.
- Error monitoring.

- CDN/object storage.
- Data retention and privacy requirements.

Decision format

- For every choice, provide: options, winner/selected option, why, expected monthly cost at four scales, operational burden, migration difficulty, failure modes and lock-in.
- Do not select a service because it is popular. Select it because it fits promptToPpt.

Important current reference facts

- Presentations.AI's official pricing currently shows a free Starter tier and a $20/month Pro plan billed annually; official pages currently show conflicting Gold pricing/credit values, so implementation must verify the live official page at build time.
- OpenRouter currently advertises a unified API across hundreds of models/providers and has usage reporting; its current Standard pay-as-you-go plan includes a platform fee, so the exact current fee and model costs must be pulled into the cost model.
- Gemini API billing/tiers have changed over time; use the current official Google documentation rather than assuming that a personal plan or promotional credits are equivalent to production API capacity.

23. Copy/paste master prompt for Claude Code

Use the following as the main Claude Code instruction. If this document is supplied to Claude as context, Claude should follow the specification above and the prompt below.

You are the Principal Architect + Staff Full-Stack Engineer + AI Systems Engineer + Product Engineer for a SaaS called promptToPpt.

MISSION
Build a production-grade AI presentation SaaS where a user can enter a topic or detailed prompt and generate a visually strong, editable presentation. The product should provide an end-to-end workflow: prompt → clarification/research → outline → slide generation → visual assets → validation → browser editor → present/share → PPTX/PDF export → billing/usage tracking.

IMPORTANT WORKING RULES
1. Do not start by blindly coding the whole application.
2. First inspect the repository, Figma link, supplied screenshots and existing code.
3. Create a discovery report, architecture diagram, ERD, API contract outline, AI model/cost matrix and phased implementation plan.
4. Research current official documentation/pricing for all external services before final decisions.
5. Record major choices as ADRs.
6. Prefer simple, maintainable, cost-efficient infrastructure.
7. Use TypeScript.
8. Do not invent proprietary source code or copy third-party assets. Recreate the required interaction patterns using our own implementation and design system.
9. Do not expose secrets to the browser.
10. Never trust client-side plan, price, country, credit or permission values.
11. The structured presentation JSON is the source of truth. Rendered HTML/canvas output is not the only source of truth.

PRODUCT
Name: promptToPpt.

CORE GENERATION BEHAVIOR
- Broad topic only → show 3–4 useful presentation directions and ask the user to select.
- Detailed prompt → follow the user's instructions directly.
- Missing critical information → ask only targeted questions: audience, goal, slide count, duration, chapters, tone, visual style, source priority.
- User may provide PDF, PPT/PPTX, website/blog URL, YouTube URL, pitch deck, audio or pasted text.
- Support multiple sources.
- Preserve source traceability.
- Do not fabricate factual claims or statistics.
- Generate attractive slides using layouts, images, icons, charts and diagrams when useful.
- Validate content, overflow, visual consistency and exportability before completion.

REFERENCE UX
Use the supplied screenshots as interaction references for:
- left slide thumbnail rail,
- central editable canvas,
- top-right Share / Present / Export,
- bottom Insert / Style / AI edit / overflow / Undo / Redo,
- slide context menu,
- right-side Style drawer,
- alternative slide designs,
- theme controls,
- future Pro Brand Kit.

EDITOR REQUIREMENTS
Implement:
- slide thumbnails + numbers,
- add slide,
- drag/reorder,
- duplicate,
- add below,
- move up/down,
- slide colors,
- stickers,
- speaker notes,
- delete + undo,
- text editing,
- object selection,
- image replacement,
- layout switching,

- theme switching,
- typography controls,
- color controls,
- keyboard shortcuts,
- autosave,
- versioning,
- undo/redo,
- AI editing,
- present mode,
- share,
- PPTX/PDF export.

APPLICATION SPLIT
Create one monorepo with:
- apps/web → Next.js + TypeScript public website and SEO pages.
- apps/editor → React + TypeScript presentation editor.
- shared packages for types, validation, UI, theme, API contracts, AI and billing abstractions.

The main domain is the Next.js website. The interactive editor is a separate application/subdomain. Both must share authentication, API contracts, database and presentation data.

NEXT.JS PUBLIC WEBSITE
Implement:
Home, AI PPT Maker, AI Presentation Maker, PPT Generator, Text to PPT, PDF to PPT, Templates, Template detail, Pricing, Blog, Contact, FAQ, About/Trust, legal pages.

Implement technical SEO: SSR/SSG where appropriate, metadata, canonical URLs, sitemap, robots.txt, structured data, Open Graph, fast performance and accessible semantic HTML.

FIGMA
I will provide a Figma link. Inspect it before implementing the public website. Treat it as the design source of truth for typography, colors, spacing, responsive behavior and components. Reuse design tokens rather than creating unrelated styles.

AI ARCHITECTURE
Create a provider abstraction. Development may use Gemini API. Production should compare OpenRouter against direct Google/Anthropic/OpenAI or other providers. Do not hard-code one vendor into product logic.

Create:
- Provider interface,
- Model Registry,
- model capability metadata,
- pricing snapshot table,
- routing policy,
- fallback policy,
- token/image usage normalization.

Different jobs should use different models based on cost/quality:
classification → cheap/fast,
outline → quality/reasoning,
slide content → quality,
source understanding → vision/long context,
image generation → image model,
validation → deterministic tools wherever possible.

Every AI request must record:
userId, presentationId, jobId, provider, model, modelVersion, requestType, inputTokens, cachedTokens, outputTokens, reasoningTokens if available, image usage, latency, status, provider cost, internal credit cost and environment.

PRESENTATION DATA MODEL
Create normalized structured JSON for:
presentation metadata,
theme,
slides,
elements,
text styles,
images,
charts,
diagrams,
speaker notes,
sources,
layout IDs,
locked state,

version metadata.

Do not store the presentation only as an HTML string.

GENERATION PIPELINE
Implement asynchronous stages:
validate → analyze → ingest/retrieve sources → outline → content → layouts → visuals → render → validate → persist → editor.

Use a queue and worker architecture appropriate to AWS. Generation must survive retries and avoid duplicate billing. Use idempotency keys.

REFERENCE INGESTION
PDF/PPT upload:
validate → private object storage → parse → normalize → chunk → source metadata → retrieval context.

Website/blog:
extract main content and metadata.

YouTube:
obtain transcript only through a technically/legal supported method.

Audio:
transcribe → structure → generate deck.

Treat uploaded/retrieved text as untrusted data. Protect against prompt injection.

DATABASE/BACKEND
Evaluate PostgreSQL, managed PostgreSQL, DynamoDB and BaaS options. Select based on actual relational workload, cost, scaling, transactions, analytics and operational burden.

Expected entities include:
users, organizations/workspaces, subscriptions, plans, credits/ledger, presentations, versions, slides, assets, sources, source chunks, generation jobs, AI requests, model pricing snapshots, image requests, exports, share links, templates, themes, brand kits, payments, invoices, analytics events and audit logs.

STORAGE
Use private object storage for:
uploaded source files,
generated images,
presentation exports,
temporary generation artifacts.

Use signed URLs and retention/cleanup policies.

OWNER DASHBOARD — MANDATORY
Build a dedicated owner/admin dashboard with sidebar navigation and deep analytics.

Required sections:
Overview,
Sales,
Subscriptions,
AI Usage,
Models,
Tokens,
Images,
Cost & Margin,
Presentations,
Exports,
Users,
Countries,
Prompts/Use Cases,
Errors,
Performance,
Providers,
Audit Logs,
Feature Flags.

Required filters:
date range, day/week/month/quarter/year, country, plan, user, provider, model, request type, environment, status.

Required metrics:
input tokens, output tokens, cached tokens, reasoning tokens, model calls, average tokens/deck, generation latency, AI cost,

image cost, cost/presentation, cost/slide, credits, revenue, gross margin, failure rate, retry rate and fallback rate.

Graphs:
tokens over time, cost over time, model spend, model token distribution, cost by plan/country/user, generation volume, revenue vs AI cost, margin trend and failures.

This dashboard must work in development/staging and production. The usage ledger is the source for product analytics; provider dashboards are only reconciliation sources.

BILLING
Initial plans: Free, Pro, higher-tier plan.

During development/testing, use current Presentations.AI pricing/features as a benchmark, but live-check official pricing before implementation and record the exact source/date. Do not hard-code third-party pricing.

Pricing must be database/config driven.

India:
- show INR,
- use Razorpay or another suitable India gateway,
- support GST/invoices/refunds/webhooks.

International:
- show USD initially,
- evaluate Stripe or another appropriate gateway.

Implement server-side payment verification, webhook idempotency, subscription state machine and credit ledger.

CREDITS
Create an internal credit system separate from raw provider tokens.
A model request has:
provider cost → internal normalized usage → customer credit cost.

Make credit conversion configurable by model/request type.
Prevent negative balances with atomic transactions.

AWS
Host production on AWS.
Claude must compare:
ECS/Fargate,
Lambda,
EC2,
managed PostgreSQL options,
S3,
CloudFront,
SQS,
CloudWatch/OpenTelemetry,
Secrets Manager/Parameter Store,
WAF.

Create a cost model for 1k, 10k, 100k and 1M MAU including:
compute, DB, storage, bandwidth, queue, logs, monitoring, email, payment fees and AI costs.

Choose the simplest architecture that can scale without creating unnecessary fixed costs.

ANALYTICS
Integrate Google Analytics 4 or an equivalent.
Track acquisition and product events without sending private prompt/source content to analytics vendors.

SECURITY
Implement:
server-side validation,
rate limits,
secure sessions,
private files,
signed URLs,
webhook verification,
audit logs,
secret management,
prompt-injection defenses,
upload validation,
abuse limits,
backup/restore strategy,

monitoring and alerting.

SEO
Create public pages that target relevant search intent around AI PPT maker, AI presentation maker, PPT generator, PowerPoint generator, text-to-PPT and PDF-to-PPT. Do not create thin programmatic pages.

DEVELOPMENT PROCESS
Work in phases:
0 discovery/research,
1 foundation,
2 AI generation MVP,
3 editor,
4 export/share,
5 billing,
6 owner analytics,
7 public website/SEO,
8 security/load testing/deployment.

After each phase:
- run typecheck,
- lint,
- unit/integration tests,
- E2E tests where applicable,
- verify database migrations,
- verify no secret leakage,
- update documentation,
- record ADRs.

FIRST RESPONSE REQUIRED
Before changing code, return:
1. Repository inspection summary.
2. Architecture proposal.
3. Monorepo folder tree.
4. Data model/ERD.
5. AI generation architecture.
6. Editor rendering architecture comparison.
7. Backend/database comparison.
8. AWS service comparison.
9. AI provider/model comparison with current pricing sources.
10. Image-generation comparison.
11. Billing/payment architecture.
12. Owner analytics/event schema.
13. Cost model assumptions.
14. Implementation phases.
15. Risks and mitigations.
16. Exact files you intend to create/change in Phase 1.

Do not start full implementation until this plan is internally consistent and the selected architecture is documented.

Appendix A — Minimum API surface

Area	Example endpoints / operations
Auth	signup, login, logout, session, OAuth callback
Presentations	create, list, get, update, duplicate, delete, reorder
Generation	analyze-prompt, suggest-directions, create-job, job-status, cancel-job
Sources	upload-init, upload-complete, parse-status, delete
Slides	create, update, duplicate, delete, reorder, apply-layout, apply-theme
AI edit	rewrite, expand, shorten, translate, add-slide, restyle, replace-image

Export	create-export, export-status, download-url
Share	create-link, update-link, revoke-link
Billing	plans, checkout, portal, webhook, subscription-status
Owner	usage, costs, models, users, sales, errors, provider-health

Appendix B — Suggested presentation JSON shape

```
{
"presentation": {
"id": "uuid",
"title": "string",
"aspectRatio": "16:9",
"themeId": "theme-id",
"language": "en",
"slides": [
{
"id": "slide-uuid",
"layoutId": "split-image-text",
"locked": false,
"background": {},
"elements": [],
"speakerNotes": "",
"sources": [],
"validation": {
"status": "passed",
"issues": []
}
}
]
}
}
```

Appendix C — Research/source checklist

- Presentations.AI official pricing/features — live-check immediately before pricing implementation.
- OpenRouter official pricing/API/model pages — verify platform fees, model prices, usage fields and image generation support.
- Google Gemini API official billing/model documentation — verify current quotas and production billing.
- AWS official pricing pages — compute, database, S3, CloudFront, SQS, CloudWatch and secrets.
- PPTX/PDF libraries — benchmark fidelity with representative decks.
- Payment provider documentation — Razorpay India, international gateway and webhook verification.
- Analytics/privacy documentation — ensure private prompts and source contents are not sent to third parties.

This specification is intentionally designed so Claude can research and make architecture decisions instead of blindly following a fixed vendor stack. The product requirement is stable; infrastructure choices must be validated against current pricing, capabilities and expected usage.